

Hands-On ML Observability with Kubernetes, Prometheus, and Grafana

Overview

In this lecture, you will build a complete ML observability pipeline using:

- FastAPI ML model
- Docker
- Kubernetes
- Prometheus
- Grafana
- Helm

You will:

- Deploy a model API
- Expose metrics
- Collect metrics using Prometheus
- Visualize metrics using Grafana

Everything will run locally.

Prerequisites (Run on LOCAL MACHINE)

Make sure you have:

- Docker installed
- Kubernetes enabled (Docker Desktop or Minikube)
- kubectl installed
- Helm installed

Verify installations:

```
docker --version
kubectl version --client
helm version
```

Step 1: Create ML API with Metrics (LOCAL)

Create a folder:

```
mkdir ml-observability-demo
cd ml-observability-demo
```

Create app.py:

```
from fastapi import FastAPI
import time
import random

from prometheus_client import Counter, Histogram, generate_latest
from fastapi.responses import Response

app = FastAPI()

REQUEST_COUNT = Counter('request_count', 'Total requests')
REQUEST_LATENCY = Histogram('request_latency_seconds', 'Request latency')

# Default root endpoint
@app.get("/")
def read_root():
    return {"message": "Welcome to the API"}

@app.get("/predict")
def predict():
    start_time = time.time()

    # Simulated prediction
    prediction = random.choice([0, 1])

    REQUEST_COUNT.inc()
    REQUEST_LATENCY.observe(time.time() - start_time)

    return {"prediction": prediction}

@app.get("/metrics")
def metrics():
    return Response(generate_latest(), media_type="text/plain")
```

Explanation

The Prometheus-related portion of this application is responsible for defining, recording, and exposing metrics so that they can be collected externally. First, the code imports `Counter`, `Histogram`, and `generate_latest` from the Prometheus client library, where a `Counter` is used to track how many times an event occurs and a `Histogram` is used to measure the distribution of values such as request latency. The line `REQUEST_COUNT = Counter('request_count', 'Total requests')` creates a counter metric that increments every time a prediction request is made, and although it is defined as `request_count`, Prometheus internally exposes it as `request_count_total`. Similarly, `REQUEST_LATENCY = Histogram('request_latency_seconds', 'Request latency')` records how long each request takes, and when `REQUEST_LATENCY.observe(value)` is called, it contributes to

multiple derived metrics such as `_bucket`, `_sum`, and `_count`, which together represent a full distribution of latency values. Inside the `/predict` endpoint, the application explicitly updates these metrics by calling `REQUEST_COUNT.inc()` to increment the total request count and `REQUEST_LATENCY.observe(time.time() - start_time)` to record how long the request took, emphasizing that metrics are not collected automatically but must be instrumented manually in the code. Finally, the `/metrics` endpoint exposes all recorded metrics by returning the output of `generate_latest()` in plain text format, which is required because Prometheus follows a pull-based model where it periodically sends HTTP requests to this endpoint to scrape the latest metric values. This design means that your application acts as a metric producer, continuously updating internal counters and histograms, while Prometheus acts as a collector that retrieves and stores these values over time for further querying and visualization in tools such as Grafana.

Step 2: Create Requirements

```
fastapi
uvicorn
prometheus_client
```

Save as `requirements.txt`

Step 3: Create Dockerfile

```
FROM python:3.10

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 4: Build and Test Docker (LOCAL)

```
docker build -t ml-api:1.0 .
docker run -p 8000:8000 ml-api:1.0
```

Test:

```
curl http://localhost:8000/predict
curl http://localhost:8000/metrics
```

Once locally tested, stop the container.

Step 5: Kubernetes Deployment

Start minikube and build the docker image for minikube to use:

```
minikube start
minikube image build -t ml-api:1.0 .
```

Create deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-api-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ml-api
  template:
    metadata:
      labels:
        app: ml-api
    spec:
      containers:
        - name: ml-api
          image: ml-api:1.0
          imagePullPolicy: Never
          ports:
            - containerPort: 8000
```

The line `containerPort: 8000` specifies the port on which your application is running *inside the container*. In your case, the FastAPI application is started with Uvicorn using port 8000 (as defined in your Dockerfile with `--port 8000`), so this tells Kubernetes: “this container listens for incoming traffic on port 8000.” It is important to understand that this port is not automatically exposed outside the pod or the cluster; it is only an internal declaration used by Kubernetes for documentation and for connecting other resources like Services. When you later define a Service with `targetPort: 8000`, Kubernetes uses this value to forward traffic from the Service to the correct port inside the container. So, `containerPort: 8000` is essentially telling Kubernetes where your application is listening within the pod, and it must match the port your application actually uses.

Apply:

```
kubectl apply -f deployment.yaml
```

Step 6: Create Service

```
apiVersion: v1
kind: Service
metadata:
  name: ml-api-service
spec:
  type: NodePort
  selector:
    app: ml-api
  ports:
```

```
- port: 8000
  targetPort: 8000
  nodePort: 30007
```

This Service configuration defines how traffic is routed to your application running inside Kubernetes. The line `kind: Service` means you are creating a network abstraction that allows external or internal clients to communicate with your pods. The `selector: app: ml-api` is critical because it tells Kubernetes to route traffic only to pods whose labels match `app=ml-api`, which must exactly match the labels defined in your Deployment; otherwise, the Service will not connect to any pods. The `type: NodePort` exposes your application outside the cluster by opening a port on each node's IP address. Inside the `ports` section, `port: 8000` defines the port that the Service itself listens on within the cluster, while `targetPort: 8000` specifies the port inside the container (matching `containerPort: 8000`) where the traffic should be forwarded. Finally, `nodePort: 30007` is the external port opened on the node, meaning you can access your application from outside the cluster using `http://<node-ip>:30007`. In summary, traffic flows as follows: a request comes to the node on port 30007, Kubernetes forwards it to the Service on port 8000, and then the Service routes it to the container's port 8000 in the appropriate pod.

Apply:

```
kubectl apply -f service.yaml
```

Access:

```
minikube service ml-api-service --url
```

Then try the endpoints on the browser, like:

```
http://127.0.0.1:59655/predict
```

Step 7: Install Prometheus using Helm

Add repo:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Install:

```
helm install prometheus prometheus-community/prometheus
```

Explanation

- Helm installs a full Prometheus stack
- Includes server, config, and services

Step 8: Configure Prometheus to Scrape API

Edit config:

Either do it using vim or nano:

```
kubectl edit configmap prometheus-server
```

Or save it on a yaml file and edit there and apply it again.

```
kubectl get configmap prometheus-server -o yaml > prometheus.yaml
```

Add to prometheus.yaml file, under scrape_configs:

```
- job_name: 'ml-api'
  static_configs:
  - targets: ['ml-api-service:8000']
```

The provided YAML defines the configuration for a Prometheus server, specifying how it collects and manages metrics within the system. Under `apiVersion: v1`, the configuration is stored inside a Kubernetes ConfigMap, where the `data` section contains multiple configuration files used by Prometheus. The fields `alerting_rules.yml` and `alerts` are currently empty, meaning no alerting rules are defined, while `allow-snippet-annotations: "false"` disables certain dynamic annotation features for safety. The main configuration is inside `prometheus.yml`, where the `global` section defines default scraping behavior: `scrape_interval: 1m` means Prometheus collects metrics every one minute, `evaluation_interval: 1m` controls how often rules are evaluated, and `scrape_timeout: 10s` sets the maximum time Prometheus waits for a response from a target. The `scrape_configs` section defines what Prometheus should monitor; the first job, `job_name: 'ml-api'`, uses a simple `static_configs` approach where Prometheus directly queries the endpoint `ml-api-service:8000`, which relies on Kubernetes internal DNS to resolve the Service name to the correct pod, and by default Prometheus will request the `/metrics` endpoint on that target.

Then apply the yaml file:

```
kubectl apply -f prometheus.yaml
```

Restart:

```
kubectl rollout restart deployment prometheus-server
```

The command `kubectl rollout restart deployment prometheus-server` forces Kubernetes to restart all the pods managed by the `prometheus-server` Deployment without changing its configuration. Internally, Kubernetes achieves this by updating the Deployment's pod template metadata (typically by adding a new timestamp annotation), which triggers a rolling update. As a result, new pods are created one by one using the same configuration, while the old pods are gradually terminated. This ensures that there is minimal downtime and that the service remains available during the restart process. In this context, you use this command after modifying the Prometheus ConfigMap because the running Prometheus pods do not automatically reload configuration changes; restarting the Deployment ensures that the pods are recreated and start with the updated configuration, allowing Prometheus to pick up new scrape targets or settings.

Step 9: Access Prometheus

```
kubectl port-forward svc/prometheus-server 9090:80
```

The command `kubectl port-forward svc/prometheus-server 9090:80` creates a temporary tunnel between your local machine and a Kubernetes Service so that you can access it as if it were running locally. Specifically, `svc/prometheus-server` refers to the Prometheus Service inside the cluster, and the mapping `9090:80` means that requests sent to port 9090 on your local machine will be forwarded to port 80 of the Prometheus Service inside Kubernetes. This is necessary because

Kubernetes Services are not always directly accessible from outside the cluster, especially when using internal service types like ClusterIP. By running this command, you effectively bypass external networking limitations and can open Prometheus in your browser at `http://localhost:9090`. The connection remains active only while the command is running, meaning it acts as a live proxy that forwards traffic from your local environment into the cluster, allowing you to interact with internal services without modifying their exposure configuration.

Open:

```
http://localhost:9090
```

Search:

- `request_count_total`
- `request_latency_seconds`

Step 10: Install Grafana

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
helm install grafana grafana/grafana
```

Get password:

```
kubectl get secret grafana -o jsonpath="{.data.admin-password}" | base64 --decode
```

Port forward:

```
kubectl port-forward svc/grafana 3000:80
```

Open:

```
http://localhost:3000
```

Step 11: Connect Grafana to Prometheus

In Grafana:

- Go to Data Sources
- Add Prometheus
- URL: `http://prometheus-server`

Step 12: Create Dashboard

Add panels and select metrics and see the graphs

- `request_count`
- `rate(request_count[1m])`
- `request_latency_seconds`

Observe Grafana updating in real time.

End of Demo

You have successfully:

- Deployed an ML API
- Exposed metrics
- Collected metrics using Prometheus
- Visualized metrics using Grafana